

MINI PROJECT REPORT

Local GitHub Repository Code Explainer

Course: Generative AI | Semester 5

1. Objective

The objective of this mini-project is to build a complete local GenAI application that accepts any public GitHub repository URL and automatically produces a simple-language, structured explanation of the codebase. The application demonstrates the complete end-to-end GenAI lifecycle running locally on the user's machine:

GitHub Repository → Code Processing → Local LLM → Backend API → Frontend UI → Codebase Explanation

2. Technology Stack

Component	Technology Used	Role & Description
Local GenAI	Hugging Face Transformers / PyTorch	Runs local open-source LLMs (Qwen 2.5 0.5B, SmolLM2, or Ollama) directly on CPU/GPU without cloud API dependencies.
Backend	FastAPI, Uvicorn, Pydantic	Provides REST API endpoints for repository analysis, input validation, and asynchronous orchestration.
Repository Processing	GitPython & Python OS Handling	Performs shallow repository cloning (depth=1), directory scanning, file filtering, and code snippet extraction.
Frontend	Streamlit	Provides an interactive web dashboard for user inputs, live status tracking, file tree viewing, and markdown export.

3. Step-by-Step Implementation

The system implements the required core features through five distinct stages:

Step 1: Shallow Repository Cloning (GitPython)

The user enters a GitHub URL (e.g., <https://github.com/username/repository>). The backend uses GitPython to perform a shallow clone (depth=1) into a local cache. This downloads only the latest commit, minimizing network bandwidth and disk usage.

Step 2: File Filtering & Priority Extraction

To fit within the local LLM's context window, the processor ignores binary files (images, videos, executables), virtual environments (venv, __pycache__, node_modules), and lock files (package-lock.json, poetry.lock). It builds an ASCII directory tree and extracts key files based on priority: README files, configuration files (requirements.txt, package.json), and entrypoint source files (app.py, main.py, index.js).

Step 3: Local LLM Prompting & Inference

The assembled repository structure, dependency list, and source code excerpts are formatted into an instruction prompt. The local LLM (Qwen/Qwen2.5-0.5B-Instruct) generates a structured, plain-English summary covering Project Overview, Key Features, Main Technologies, and How It Works.

Step 4: FastAPI Backend Service

FastAPI coordinates the cloning, processing, and LLM inference. It exposes endpoints: POST /api/explain (full analysis), GET /api/models (available models), and GET /api/health (server and hardware status).

Step 5: Streamlit Frontend Display

The Streamlit frontend presents an intuitive interface where users can enter a URL or choose a sample repository. It renders the explanation, provides an ASCII tree view, allows viewing individual source code files with syntax highlighting, and provides a button to download the explanation as Markdown.

4. Example Input & Output

Input: <https://github.com/streamlit/demo-uber-nyc-pickups>

Generated Local LLM Explanation:

Project Overview:

This project is an interactive data visualization web application that analyzes and maps historical Uber taxi pickup patterns across New York City.

Key Features:

The application allows users to:

- View and interact with Uber pickup data on a dynamic 3D map.
- Filter pickup activity by specific hours of the day using interactive sliders.
- Visualize geographic pickup density using hexagon layers.
- Cache large datasets locally for fast user interactions.

Main Technologies:

- **Python:** Core application programming language.
- **Streamlit:** Web frontend framework and reactive UI components.
- **PyDeck & Altair:** 3D geospatial mapping and statistical data charts.
- **Pandas & NumPy:** Data manipulation, filtering, and numerical processing.

How It Works:

1. The user opens the web application and selects an hour on the Streamlit slider widget.
2. The Python backend filters the raw Uber pickup records for the selected time window.
3. PyDeck computes coordinates and renders the geographic density map.
4. The frontend displays the updated map and summary charts directly to the user.

5. Execution Instructions

The application can be started using the following simple steps:

1. **Install Dependencies:** `pip install -r requirements.txt`
2. **Start Backend:** `python run_backend.py` (runs on `http://127.0.0.1:8000`)
3. **Start Frontend:** `python run_frontend.py` (opens in browser at `http://localhost:8501`)
4. **Or One-Click Launch (Windows):** Double-click `run_all.bat`

6. Conclusion

This project successfully fulfills the requirement of building a local, private GenAI tool for code explanation. By combining shallow git cloning, intelligent file filtering, and small open-source language models running locally, the system provides quick and accurate summaries of unfamiliar codebases without requiring paid APIs or internet connectivity during inference.